

Fluxos de Trabalho Git Modernos

Estratégias para Colaboração, Qualidade e Implantação de Código

INSTITUIÇÃO EXECUTORA

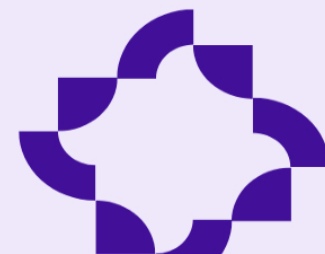


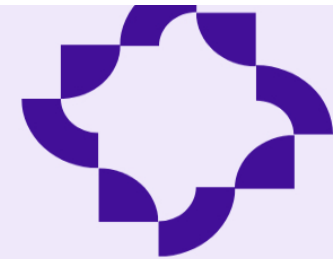
COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Sumário

- Introdução aos Fluxos de Trabalho
- Git Flow: Estrutura e Processo
- Trunk-Based Development: Agilidade
- O Papel Essencial dos Pull Requests
- Comparativo e Aplicações Práticas



INSTITUIÇÃO EXECUTORA



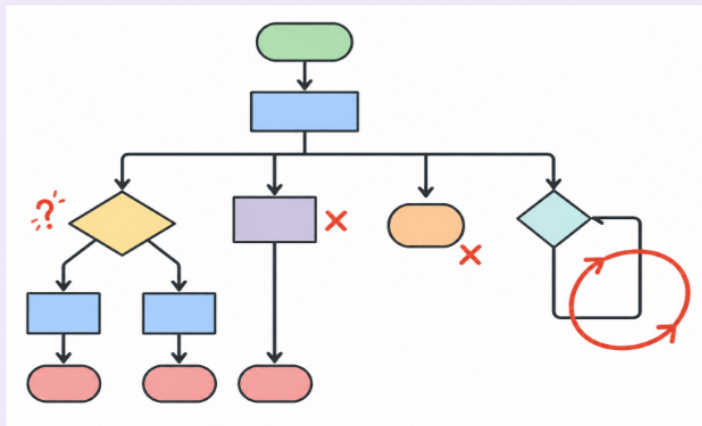
COORDENADORA



APOIO

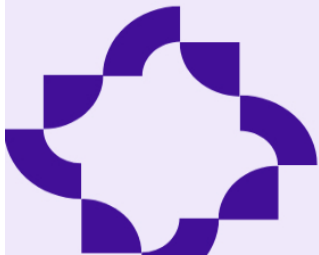


Gestão de Fluxos Modernos



Fluxogramas ilustram caminhos, erros e loops em fluxos de trabalho.

Os fluxos de trabalho em sistemas de controle de mudanças são estratégias padronizadas para gerenciar o versionamento de código, sendo cruciais para a **colaboração eficiente** entre desenvolvedores, a **auditoria e qualidade** através de revisões e a promoção de mudanças para **implantação**. A escolha do fluxo correto impacta diretamente a velocidade de entrega (Lead Time) e a estabilidade do sistema.



INSTITUIÇÃO EXECUTORA

 **aponti**

 **MCTI
FUTURO**

COORDENADORA

 **Softex**

APOIO

**MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO**

**GOVERNO DO
BRASIL**



Git Flow: Estrutura Rígida

O Git Flow é um modelo estruturado com ramificações (branches) de longa duração. Ideal para projetos com ciclos de lançamento bem definidos, isola o código de produção do desenvolvimento ativo, garantindo estabilidade. Permite o processo paralelo de preparação de novas versões enquanto correções são implementadas, mas exige alta disciplina da equipe.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Branches de Longa Duração (Git Flow)

Branch Main (Master)

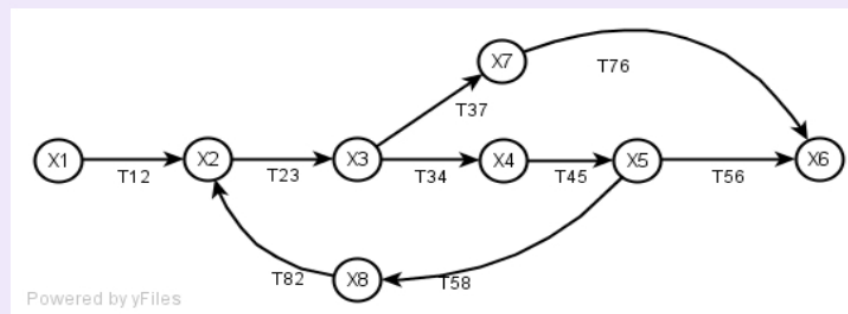
Contém o código em **estado de produção**. Cada *commit* nesta branch representa uma nova versão estável do software, pronta para *deploy*.

Branch Develop

Funciona como a branch de **integração principal**. É onde os desenvolvedores unem suas novas funcionalidades terminadas para testes de integração abrangentes.

Feature Branches

Criadas a partir da `develop` para o desenvolvimento de funcionalidades **específicas**. Quando concluídas e testadas, retornam para a `develop` e são deletadas.



Fluxo de sinal: integração e ramificações em fluxos de trabalho.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Branches de Release e Hotfix

Esta tabela descreve as características e funções das branches de Release e Hotfix no fluxo de trabalho

Branch	Origem	Destino	Propósito
Release	develop	main e develop	Preparação final para lançamento, ajustes e correção de bugs de última hora.
Hotfix	main	main e develop	Correção urgente de bugs críticos encontrados diretamente em produção.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Avaliação do Git Flow



Vantagens Chave

Excelente para equipes grandes e projetos que suportam múltiplas versões simultâneas. Garante **auditoria rigorosa** através do isolamento de branches. É ideal para lançamentos agendados e modelos de desenvolvimento tradicionais



Desvantagens Apontadas

O processo é **burocrático** e mais lento. Pode dificultar a Integração Contínua (CI) devido ao longo tempo de vida das branches. Existe alto risco de '*Merge Hell*' (conflitos complexos) ao integrar grandes volumes de código.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Trunk-Based Development (TBD)

Diferente do Git Flow, o Trunk-Based Development (TBD) foca na simplicidade e velocidade, com todos os desenvolvedores integrando suas mudanças diretamente em uma única *branch* principal, conhecida como **Trunk** (geralmente a `'main'`). As integrações são frequentes, ocorrendo várias vezes ao dia, com a filosofia de minimizar a divergência de código. Este modelo exige alta maturidade técnica e testes automatizados robustos para garantir a estabilidade do Trunk.



Ambiente de trabalho colaborativo, essencial para desenvolvimento ágil.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





O Processo de Integração no TBD

1

Sincronizar com o Trunk

O desenvolvedor inicia o trabalho baixando a versão mais recente do *Trunk* para garantir que sua base de código esteja atualizada com as últimas modificações.

2

Criar *Short-Lived Branches*

Ramificações são criadas para as novas funcionalidades ou correções, com uma duração máxima de **1 a 3 dias** para manter o ciclo de desenvolvimento ágil e evitar divergências prolongadas.

3

Executar Testes Locais

Antes de qualquer *commit*, o desenvolvedor executa uma suíte de testes automatizados localmente, garantindo que as alterações não introduzam regressões e que o código esteja funcional.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





O Processo de Integração no TBD

4

Realizar *Merge* Frequente

O código é integrado de volta ao *Trunk* o mais rápido possível após os testes locais passarem, minimizando o risco de conflitos complexos e mantendo a base de código unificada.

5

Receber *Feedback* Contínuo

Após o *merge*, o pipeline de Integração Contínua/Entrega Contínua (CI/CD) é acionado, validando a integração imediatamente e fornecendo *feedback* rápido sobre a qualidade do código.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Feature Flags

Feature Flags (Alternadores de Funcionalidade) são chaves lógicas no código que permitem ativar ou desativar funcionalidades em tempo de execução. Isso possibilita a implantação de código incompleto no Trunk sem que ele seja visível para o usuário final, e facilita o rollback imediato em produção ao desativar a flag em caso de erro.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Avaliação do Desenvolvimento Baseado em Trunk



Vantagens

O Trunk-Based Development (TBD) acelera drasticamente o ciclo de feedback, permitindo que as alterações sejam integradas e testadas rapidamente. Ele elimina *conflitos de merge* complexos, um problema comum em outras estratégias de ramificação. Além disso, o TBD está perfeitamente alinhado com as práticas de **DevOps** e **CI/CD** (Integração Contínua/Entrega Contínua), otimizando a entrega de software.



Desvantagens

O TBD exige desenvolvedores **experientes e seniores** devido à disciplina e responsabilidade necessárias. Há uma dependência crítica da *automação de testes*; se os testes falharem, o `Trunk` (ramo principal) fica instável, interrompendo o desenvolvimento. Requer disciplina extrema para manter o `Trunk` sempre estável e verde, garantindo que novas funcionalidades não introduzam regressões.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Git Flow vs. Trunk-Based

Esta tabela compara as principais características dos fluxos de trabalho Git Flow e Trunk-Based Development (TBD).

Característica	Git Flow	Trunk-Based
Foco	Controle e Isolamento	Velocidade e Agilidade
Tamanho da Branch	Longa duração (semanas)	Curta duração (horas/dias)
Integração	Eventual (Merge final)	Contínua (Várias vezes ao dia)
Risco de Conflito	Alto	Baixo
Complexidade	Alta (Muitas branches)	Baixa (Foco no Trunk)
CI/CD	Difícil de implementar	Nativo ao modelo



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





O Papel dos Pull Requests

Notificação

Avisa formalmente à equipe que um trabalho está pronto para revisão, iniciando o processo colaborativo.

Code Review

O coração do PR. Permite que pares revisem o código, sugiram melhorias e garantam padrões de design antes da integração.

Histórico

Cria uma trilha de auditoria valiosa, registrando por que cada mudança foi feita e quem a aprovou.



INSTITUIÇÃO EXECUTORA



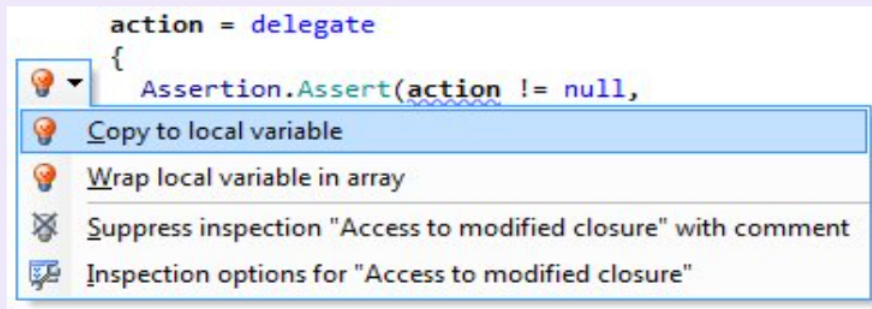
COORDENADORA



APOIO



PRs e Automação de CI



ReSharper sugere melhorias para análise estática de código.

- **Pipelines de CI:** O Pull Request dispara automaticamente testes unitários, análise estática de código (linters) e varreduras de segurança.
- **Bloqueio de Merge:** O sistema pode ser configurado para impedir o merge do código se os testes falharem ou se não houver um número mínimo de aprovações humanas.
- **Conformidade:** Facilita a adesão a normas técnicas, garantindo que nenhum código entre no ambiente principal sem validação automatizada.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Qual fluxo de trabalho é mais indicado para uma equipe que visa implementar Continuous Deployment (CD), realizando múltiplas implantações em produção por dia e utilizando Feature Flags para ocultar funcionalidades incompletas?

- A Hotfix Branching
- B Git Flow
- C Trunk-Based Development
- D Pull Request isolado



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Qual fluxo de trabalho é mais indicado para uma equipe que visa implementar Continuous Deployment (CD), realizando múltiplas implantações em produção por dia e utilizando Feature Flags para ocultar funcionalidades incompletas?

- ✗ Hotfix Branching
- ✗ Git Flow
- ✓ Trunk-Based Development
- ✗ Pull Request isolado



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





 Discuta com o colega ao lado

Qual dos modelos (Git Flow ou Trunk-Based Development) seria mais desafiador de implementar em uma empresa migrando de processos manuais para automatizados?



Considerem a burocracia do Git Flow versus a agilidade do Trunk-Based Development e justifiquem suas escolhas.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Conclusão

- Git Flow oferece estabilidade com branches longas para lançamentos agendados, mas gera burocracia e potenciais conflitos de merge.
- Trunk-Based Development prioriza integração contínua na branch principal, exigindo Feature Flags e testes robustos para estabilidade.
- Pull Requests centralizam revisão de código por pares e gatilham automação de CI/CD, assegurando qualidade antes do merge.
- A escolha do fluxo impacta diretamente Lead Time e estabilidade; TBD é ideal para CD, enquanto Git Flow para ciclos mais longos.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

